

第 06 章 | 機器學習模型評估與超參數調校

學生版公開講義 | 請搭配本課程 Colab，依照段落逐步操作與自我檢查。

來源說明：課程參考《Python Machine Learning, 3rd Edition》；原始範例程式碼採 MIT License。本檔由恩恩統計家教整理為學生版學習摘要，不含原書 PDF 或掃描內容。

1. 本章學習路線

模型能在訓練資料上運作，不代表能泛化到新資料。本章建立可重複的評估與超參數搜尋流程，用交叉驗證比較模型，並辨識過擬合與資料外洩。

適合對象：已掌握基礎模型呼叫，希望自動化調優流程並確保評估客觀性的資料科學家。

先備知識：熟悉 `train_test_split`、基礎前處理（如 `StandardScaler`）以及過擬合與欠擬合的基礎定義。

學習目標：

建構 Pipeline (管線) 封裝工作流，從架構面上徹底杜絕「資料外洩」。

實作 k-fold 交叉驗證 與 分層抽樣 (Stratified)，取得穩定且無偏的效能估計。

利用 學習曲線與驗證曲線 診斷模型病灶（高偏差 vs. 高變異）。

掌握 Grid Search (網格搜索) 與 Nested CV (巢狀交叉驗證) 自動化調參。

解析 混淆矩陣、F1-score (調和平均數) 與 ROC/AUC (門檻移動分析)。

實作類別不平衡資料的處理策略（權重調整與重採樣）。

掌握了這些科學評估標準，我們才能具備足夠的資料信心，將模型推向生產環境。

2. 核心觀念：白話解讀三大支柱

在比較模型前，先把資料前處理、訓練與評估固定成可重複執行的流程，避免手動步驟造成資料外洩或評估偏差。

Pipeline (管線)：把前處理與模型依固定順序封裝在同一流程中。交叉驗證時，每一折只用該折訓練資料估計前處理參數，可降低資料外洩風險。

k-fold 交叉驗證 —— 「輪流當模擬考卷」：「只做一次 `train_test_split` 就像一輩子只考一次期末考，成績好壞可能純屬運氣。k-fold 是將題庫切成 k 份，輪流拿 1 份當考試、其餘當練習，重複 k 次。這樣算出的平均分與標準差，才代表模型真正的學力穩定度。」

Grid Search (網格搜尋)：列出候選超參數組合，對每一組執行交叉驗證，再依指定評估指標選出表現較佳的設定。

3. 重要術語中英對照表

英文術語	繁體中文翻譯	一句話核心定義
Holdout Method	留出法	將資料簡單切分為訓練、驗證與測試集的方法。
k-fold Cross-Validation	k 折交叉驗證	將資料均分 k 份輪流驗證，以取得低變異的效能估計。
Hyperparameters	超參數	訓練前需手動設定的參數（如正規化強度 C 或 γ ）。
Learning Curve	學習曲線	以「樣本數」為 X 軸，診斷 Bias (偏差) 與 Variance (變異) 的圖表。
Validation Curve	驗證曲線	以「超參數數值」為 X 軸，尋找參數甜蜜點的圖表。
Confusion Matrix	混淆矩陣	展示 TP, FP, TN, FN 四種預測結果的對照表。
Precision / Recall	精確率 / 召回率	「查準率」（怕狼來了）與「查全率」（怕漏掉壞人）。
F1-score	F1 分數	精確率與召回率的調和平均數，平衡兩者的表現。
ROC / AUC	ROC 曲線與面積	透過移動決策門檻所得的綜合分類戰鬥力指標。
Class Imbalance	類別不平衡	某些類別樣本數遠多於其他類別（如癌症僅佔 1%）的現象。

4. 自動化工作流：Pipeline 的架構與防禦性開發

手動對整體資料進行標準化（Standardization）是新手最常犯的錯誤，這會導致測試集的統計資訊滲透進訓練流程，產生虛假的超高準確率。

Pipeline 順序：標準化 → PCA → 分類器。

流程邏輯分析

在 Pipeline 的架構中：

訓練階段 (.fit)：中間層的 Transformer 會依序執行 `fit_transform`，僅從訓練集學習參數（如平均值）。最後一關必須是 Estimator，僅執行 `fit`。

預測階段 (.predict)：測試資料僅通過中間層的 `transform`（套用訓練集學到的參數），確保測試集對模型而言是「完全陌生」的。

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression

# 宣告生產線：標準化 -> PCA 降維 -> 邏輯斯迴歸
pipe_lr = make_pipeline(StandardScaler(),
                        PCA(n_components=2),
                        LogisticRegression(random_state=1, solver='lbfgs',
                                          max_iter=10000))
```

5. 效能估計：k-fold 交叉驗證與分層抽樣

單次切分的 Holdout Method 最大的局限在於結果極度依賴切分方式。根據 Kohavi (1995) 的研究，k=10 通常是偏誤 (Bias) 與變異度 (Variance) 權衡下的最佳選擇。

分層抽樣 (Stratified) 的必要性：在分類問題中，我們必須確保每一折 (Fold) 都是母體的「代表性縮影 (Microcosm)」。如果 90% 是良性腫瘤，每一折也應維持 9:1 的比例，否則某次驗證可能完全沒有惡性樣本，導致評估失效。

```
from sklearn.model_selection import cross_val_score
# 使用 10 折交叉驗證，n_jobs=-1 開啟全核心平行運算
scores = cross_val_score(estimator=pipe_lr, X=X_train, y=y_train, cv=10, n_jobs=-1)
print(f'CV Accuracy: {scores.mean():.3f} +/- {scores.std():.3f}')
```

6. 使用學習曲線與驗證曲線診斷模型

閱讀學習曲線時，請同時比較訓練分數、驗證分數及兩者差距。

學習曲線 (Learning Curve)：

High Bias (欠擬合)：訓練與驗證準確率都很低且貼近。硬道理：此時收集更多資料也沒用，必須增加模型複雜度或減少正規化。

High Variance (過擬合)：兩線間有巨大間距。診斷：模型在死背。解決方案：增加訓練樣本或增強正規化。

驗證曲線 (Validation Curve)：X 軸為超參數（如 C）。我們在對數尺度上移動 C，尋找兩線同時維持高位且間距收攏的「甜蜜點」。

7. 自動超參數搜尋：GridSearchCV 與 Nested CV

手動調整多組參數（如 SVM 的 C 與 γ ）極其低效。

Pro-Tip (命名規則)：在 Pipeline 中調參需使用雙底線，如 `svc__C`。若不確定前綴名稱，可執行 `pipe.get_params().keys()` 檢查。

巢狀交叉驗證 (Nested CV)：當我們要在不同演算法（如 SVM vs. Decision Tree）之間做選擇時，單純的 GridSearch 會導致評估過於樂觀。我們應採用「5x2」架構：內迴圈選最佳參數，外迴圈評估該演算法的真實泛化能力。

```
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC

pipe_svc = make_pipeline(StandardScaler(), SVC(random_state=1))
param_grid = [{'svc__C': [0.1, 1, 10, 100], 'svc__kernel': ['linear']},
               {'svc__C': [0.1, 1, 10, 100], 'svc__gamma': [0.01, 0.1], 'svc__kernel':
                ['rbf']}]

gs = GridSearchCV(estimator=pipe_svc, param_grid=param_grid, scoring='accuracy', cv=2)
# Nested CV 評估演算法效能
nested_scores = cross_val_score(gs, X_train, y_train, cv=5)
```

8. 當準確率說謊：進階評估指標

在癌症預測中，99% 的準確率可能是陷阱。

混淆矩陣 (Confusion Matrix)

實際類別 / 預測類別	預測負類 (Benign)	預測正類 (Malignant)
真實負類	TN	FP (誤判為正類)
真實正類	FN (誤判為負類)	TP

指標深度解析：

F1-score：精確率與召回率的調和平均數 (Harmonic Mean)。與算術平均不同，調和平均會嚴厲懲罰極端值（如 Precision 高但 Recall 極低時，F1 會顯著下降）。

ROC 曲線與 AUC：ROC 曲線不是靜態快照，而是透過移動決策門檻 (Threshold Shifting) 從 0 到 1 描繪出的動態性能曲線。

處理類別不平衡的實戰策略：

代價敏感學習：在模型宣告時加入 `class_weight='balanced'`。

重採樣：使用 `sklearn.utils.resample` 進行向上採樣 (Upsampling) 少數類別。

9. Google Colab 實作步驟與錯誤排除

平行加速：設定 `n_jobs=-1` 以調用雲端環境的所有 CPU 核心。

常見錯誤與診斷：

ConvergenceWarning：模型不收斂。建議：`LogisticRegression(max_iter=10000)`。

運算時間：GridSearch 的組合數會隨參數候選數相乘。初次執行可先縮小 `param_range`，確認流程正確後再擴大範圍。

架構錯誤：Pipeline 最後一關必須是 Estimator，前置關卡必須具備 `transform` 方法。

10. 動手練習與自我檢查表

實作練習

參數探險：針對隨機森林模型，使用 `Validation Curve` 觀察 `n_estimators` 對過擬合的影響。

模型比較練習：實作 Nested CV，比較 SVM (RBF kernel) 與隨機森林在乳癌資料集上的表現。

自我檢查表 (Checklist)

- ☐ 我能正確使用「步驟名稱 + 雙底線」在 Pipeline 中定位超參數。
- ☐ 我了解當學習曲線顯示 High Bias 時，增加資料量並非解決方案。
- ☐ 我能說明為何 F1-score 使用 調和平均數 而非一般平均數。
- ☐ 我知道如何在 `GridSearchCV` 中設定 `refit=True` 以自動取得最佳模型。
- ☐ 面對癌症資料，我能解釋為什麼 Recall 通常比 Precision 更重要。

本章總結：本章整理交叉驗證、學習曲線、驗證曲線、超參數搜尋與模型比較。請接著完成 Colab 練習，檢查評估流程是否避免資料外洩。